



FRI ReferatBot – Domain-Specific assistant for UL FRI Student Affairs Office

Gašper Kolbezen, Samo Hribar and Gašper Suhadolnik

Povzetek

General-purpose Large Language Models (LLMs) are highly capable at open-domain conversation, but they frequently hallucinate when asked institution-specific questions they were not trained on. This project addresses that limitation by building **ReferatBot**, a domain-specific conversational assistant for the Student Affairs Office (Referat) of the Faculty of Computer and Information Science at the University of Ljubljana (UL FRI). The assistant serves the faculty's enrolled students by answering questions about enrolment procedures, exam regulations, thesis submission, student status, fees, and related administrative topics. We implemented a Retrieval-Augmented Generation (RAG) pipeline grounded in a curated corpus of 427 scraped web pages and 33 official PDF rulebooks, totalling approximately 500,000 words. This report covers the project motivation, a survey of related work, the implemented methodology, and an evaluation of the system.

Keywords

Retrieval-Augmented Generation, Large Language Models, Domain-Specific Chatbot, Student Affairs, Slovenian NLP

Advisors: Slavko Žitnik

Introduction

Students at university faculties routinely encounter administrative questions, which are answered on the UL FRI website and in official study regulations, yet students frequently contact the Student Affairs Office (Referat) by email or in person for answers that are already publicly accessible. This imposes a significant workload on administrative staff and introduces delays for students.

General-purpose LLMs such as GPT or Llama cannot always reliably answer these questions: Some UL FRI-specific rules (e.g., the precise ECTS thresholds for year advancement) or even FRI specific questions are hard for LLMs to answer correctly and the models hallucinate plausible-sounding but incorrect answers. The goal of this project was to close that gap by building a specialised assistant that retrieves factual information from a verified, institution-specific knowledge base before generating a response.

Our target domain was the UL FRI Student Affairs Office. The closest existing analogue within the University of Ljubljana is the AI student assistant deployed on the Faculty of Mechanical Engineering (UL FS) website, which is embedded as a live chat widget under the title “AI pomoč za študente UL FS”. Our system aims to replicate and extend this concept for

UL FRI, with reproducible code and open dataset.

System prompting enforces a “Student Affairs Advisor” persona with guardrails against out-of-domain questions and instructions to direct unanswerable queries to the Referat office. The system is evaluated by comparing three embedding models on a handcrafted FAQ dataset.

Related Work

Domain-Specific Knowledge Incorporation

Incorporating domain-specific knowledge into LLMs is done through the following main approaches: prompt engineering, retrieval-augmented generation (RAG) [1], and fine-tuning. Prompt engineering is the fastest and cheapest method, where carefully designed prompts provide the model with additional knowledge at runtime. While this technique is fast and quick to set up, it offers limited depth and accuracy in some cases, including domain tasks.

RAG [1] solves this issue by connecting the model to external knowledge sources, improving accuracy and transparency, though at the cost of added system complexity and latency. Knowledge is obtained from internal documents, websites, PDFs, company databases etc. and is typically stored in some vector database, such as FAISS, Pinecone, Weaviate, etc. Du-

ring inference, relevant documents are first retrieved from the database based on user prompt using vector search, and are then passed on to the underlying LLM, which generates the answer.

Fine-tuning embeds domain expertise directly into the model, delivering high performance and consistency for specialized tasks, but requires significant data, time, and ongoing maintenance. It is also harder to update or add new knowledge, which is done by additional fine-tuning of the model.

In practice, these methods are often combined, with RAG [1] providing dynamic knowledge, fine-tuning ensuring consistent behavior, and prompting orchestrating interactions.

Existing Solutions

There are many existing university chatbot solutions using RAG architecture that differ in used technologies and purpose. One example of such a system is a chatbot developed for Mississippi State University by Neupane et al., which uses BarkPlug v2 [3] architecture. For context retrieval it uses an embedding model for vectorization of external data sources and Chroma DB vector database. Actual context retrieval is done by a retriever technique provided by LangChain [4] using Maximum Marginal Relevancy and Similarity Search methods. For the underlying LLM, it utilizes OpenAI's gpt-3.5-turbo. The system achieved a mean RAGAS score of 0.96, and has proven itself to be practical and effective for real-world usage through various qualitative experiments.

Other examples of such chatbots are Meri AI [5] and AceIt-ECE-Capstone [6]. Meri AI [5] is a campus intelligence system, which runs a LangGraph workflow [4] for intent classification, route detection, context retrieval, and response generation. It uses Supabase PostgreSQL with pgvector as knowledge database containing content scraped from university resources. AceIt-ECE-Capstone [6] is a course assistant focused on study help. It uses RAG [1] pipeline where course data is embedded with Amazon Titan Text Embeddings v2 and stored in Amazon RDS PostgreSQL. It uses Llama 3.3 to generate responses based on user query and retrieved database knowledge.

Another example of a university chatbot is a virtual AI assistant developed by the Faculty of Mechanical Engineering (Fakulteta za strojništvo) at the University of Ljubljana. Unfortunately, it seems that there is no publicly available information about its architecture and even when asked the bot itself, it was unable to answer it accurately. It was, however, able to explain that the data used to provide user query with relevant context comes from UL FS webpage and standard Q&A entries.

Methods

Dataset

For the purpose of this project, we first curated a dataset. This was done by scraping 427 HTML pages from UL FRI websites and collecting 33 official PDF rulebooks. Using Microsoft's Markdown, we converted these into Markdown.

Next, we performed data sanitization by removing HTML navigation menus, stripping standalone hyperlinks, resolving PDF layout artifacts (e.g., mid-sentence line breaks), and collapsing whitespace. The final, cleaned corpus contains approximately 500,000 words of relevant administrative text.

Vector storage and retrieval

For vector storage and retrieval, we utilize TimescaleDB with the pgvector extension, managed through the `timescale-vector` Python client. Document chunks are represented as 1024-dimensional vectors generated by the BAAI/bge-m3[2] model, which was selected for its strong performance in multi-lingual tasks. To ensure efficient and scalable retrieval, we implemented a Hierarchical Navigable Small World (HNSW)[7] index using cosine similarity. The database schema stores the raw text content alongside JSONB metadata (including source file origin, chunk identifiers, and document headers) and UUIDs generated from timestamps. This architecture supports both semantic similarity search and precise metadata filtering, such as time-range queries or source-specific retrieval.

Large Language Model

The synthesis component of our RAG system is powered by the GaMS-9B-Instruct-Nemotron model, a 9-billion parameter instruct-tuned model optimized for the Slovenian language. The model is deployed locally using the Hugging Face pipeline API on NVIDIA hardware (cuda). Since the Gemma 2 chat template does not natively support a `system` role, system instructions are instead injected as the opening user turn, with the model acknowledging them in a synthetic assistant turn before the actual query is posed. This three-turn pattern reliably activates instruction-following behaviour without modifying the model weights. We employ a structured prompting strategy where the model is provided with a comprehensive system prompt that defines its role as a specialized Student Affairs Advisor for UL FRI. This prompt enforces strict operational constraints: the assistant must use the provided context exclusively, remain transparent when information is missing, maintain a professional tone, and strictly adhere to security guidelines (such as refusing to reveal internal instructions). During inference, the top-k relevant chunks retrieved from the vector store are formatted into a JSON structure and passed to the model alongside the user's query to ensure the generated responses are grounded in the verified faculty knowledge base. Listing 1 shows an excerpt of the system prompt.

Listing 1. Excerpt of the system prompt.

```
# Role and Purpose
You are an AI assistant for the study process at
FRI.
Your task is to formulate a meaningful response
based on the question asked and the context from
the knowledge base.

# Guidelines:
1. Respond in the user's language.
2. Use only information from
```

the provided context.

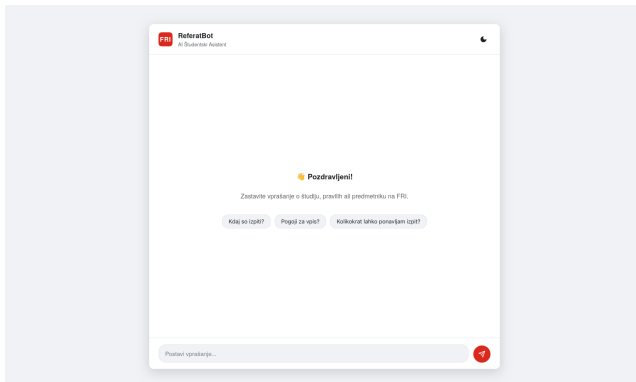
4. Be transparent when there is not enough information for a complete answer.
9. If the question is not related to studies:
"I cannot help you with matters that are not related to studies at FRI."

System Architecture

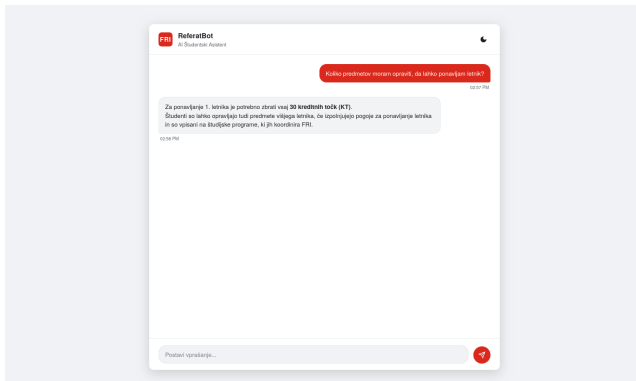
Figure 1 shows the full deployment architecture. The online path (left to right) covers query handling: the user's message travels from the browser frontend to the FastAPI backend, which embeds the query, retrieves the top- k chunks from TimescaleDB/pgvector, and passes them together with the query to the Synthesizer. The Synthesizer constructs the three-turn message list and calls GaMS-9B-Instruct-Nemotron, whose answer is returned to the user. The offline ingestion path (top) shows how the corpus of HTML pages and PDFs is processed by Markdown and the chunker before embeddings are written to the vector store.

Results

The final implementation resulted in a functional web-based conversational assistant. Figure 2 shows the initial user interface with suggested questions, while Figure 3 demonstrates a successful retrieval and synthesis cycle where the bot correctly answers a query regarding year repetition requirements.



Slika 2. ReferatBot Landing Page. The interface includes a dark mode toggle and suggested starting questions.



Slika 3. Example Conversation. The bot accurately explains the 30 ECTS requirement for repeating the first year.

Embedding Model Comparison

During development we evaluated three embedding models on our handcrafted FAQ dataset, scored by whether the correct answer appeared in the top retrieved chunks. Table 1 summarises the results.

Tabela 1. Embedding model accuracy on the FAQ evaluation dataset.

Model	Dim.	Accuracy
intfloat/multilingual-e5-base [8]	768	64%
google/gemma-embedding-300m [9]	768	77%
BAAI/bge-m3 [2]	1024	77%

Both bge-m3 [2] and gemma-embedding-300m [9] outperformed multilingual-e5-base [8] by 12 percentage. Despite same accuracy, gemma-embedding-300m requires HuggingFace authentication, whereas bge-m3 is publicly available without login, simplifying deployment and community testing. No significant latency difference was observed between the two despite bge-m3's higher dimensionality (1024 vs. 768), so we selected **BAAI/bge-m3** for the final system.

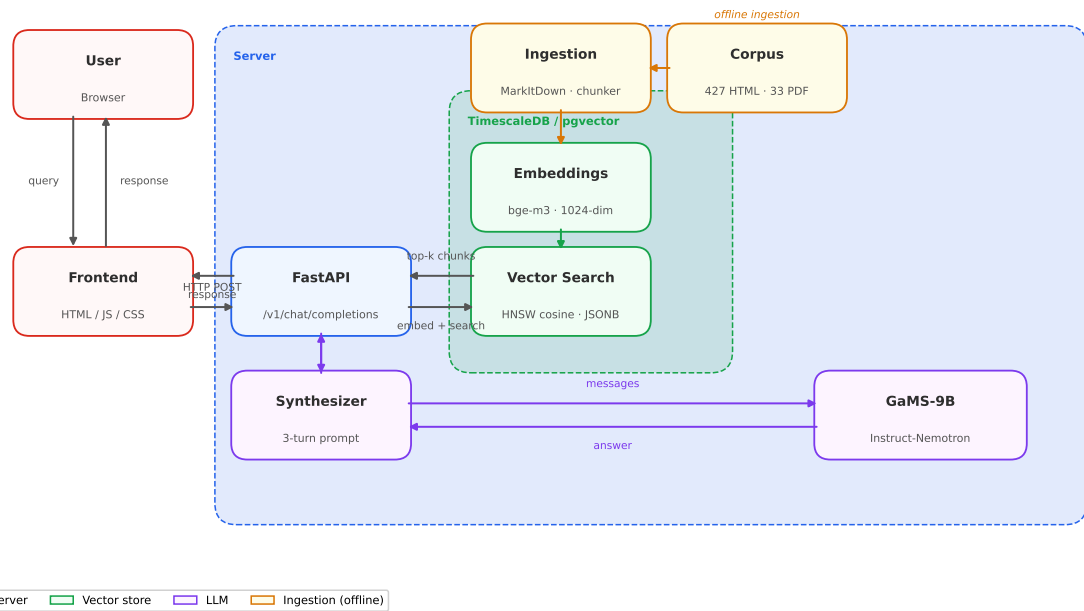
To support multi-turn dialogues, we also implemented an LLM-driven query-rewriting step prior to vector retrieval. Evaluation showed the system successfully maintained context across multi-turn follow-up questions.

RAG vs. Vanilla LLM Evaluation

To quantify the RAG pipeline's effectiveness, we evaluated the GaMS-9B model against a standalone "Vanilla" LLM baseline using a dataset of 20 domain-specific student queries. The RAG-enabled model achieved a manual correctness score of 19/20, vastly outperforming the Vanilla LLM's 4.5/20. Qualitative analysis highlighted several Vanilla LLM failure modes that our architecture resolved:

- **System Hallucinations & Program Confusion:** The Vanilla LLM frequently invented IT systems (e.g., "e-Študent", fake URLs) and fabricated program names (e.g., "informatika in mediji" for IŠRM). The RAG system correctly identified STUDIS and exact degree requirements.
- **Regulatory Inaccuracies:** Without context, the model hallucinated rules, such as a 14-day exam deregistration deadline and a requirement to pass all exams to repeat a year. RAG accurately retrieved the actual 2-day deadline and the 30 ECTS requirement.
- **False Refusals:** For highly specific factual questions (e.g., naming AI thesis mentors or exam costs), the Vanilla LLM triggered out-of-domain refusal guardrails while the RAG system successfully answered these queries.

These results confirm that while the base model possesses strong linguistic capabilities, the RAG architecture is essen-



Slika 1. ReferatBot deployment architecture. Dashed borders denote logical groups. Orange nodes are offline; all other nodes run at inference time.

tial for grounding responses in factual, institution-specific knowledge.

Discussion

Our implementation of ReferatBot demonstrates that a localized RAG pipeline is highly effective for Slovenian academic administration. However, several areas for improvement remain. The system currently relies on scraped Markdown data; establishing an automated synchronization pipeline with the faculty's official content management system would ensure regulations are strictly up to date. Furthermore, exploring smaller, highly quantized models for the query-rewriting step could further reduce response latency while maintaining conversational context.

Literatura

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- [3] Subash Neupane, Elias Hossain, Jason Keith, Himanshu Tripathi, Farbod Ghiasi, Noorbakhsh Amiri Golilarz, Amin Amirlatifi, Sudip Mittal, and Shahram Rahimi. From questions to insightful answers: Building an informed chatbot for university resources. In *2024 IEEE Frontiers in Education Conference (FIE)*, pages 1–9. IEEE, 2024.
- [4] langchain langchain. applications that can reason. powered by langchain. <https://www.langchain.com/>. Accessed: 2026-03-19.
- [5] Meri ai: Ai-powered campus navigation & intelligence system adama science and technology university. https://github.com/CSEC-ASTU/Meri_AI. Accessed: 2026-03-19.
- [6] Ace it - ai study assistant. <https://github.com/UBC-CIC/AceIT-ECE-Capstone>. Accessed: 2026-03-19.
- [7] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.
- [8] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024.
- [9] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, et al. Embeddinggemma: Powerful and lightweight text representations. 2025.

Appendix A: Embedding Evaluation Dataset

Table 2 lists the 11 handcrafted questions used to evaluate the retrieval accuracy of the three embedding models discussed in the Results section. The full dataset used for the RAG vs. Vanilla LLM evaluation, including manual annotations and failure mode logs, is publicly available in the project repository.

Tabela 2. Questions used for the embedding model comparison.

#	Question
1	According to the instructions for the Master's thesis at UL FRI, how many ECTS credits is the thesis worth, and approximately how many hours of work does this represent?
2	What is the required duration for the oral presentation during a Master's thesis defense at UL FRI?
3	To complete a doctoral program at UL FRI, what specific requirement must be met regarding scientific publications before submitting the dissertation for evaluation?
4	What is the minimum number of credits (ECTS) a doctoral student must complete to advance from the first to the second year of study?
5	Under what condition can a student apply for an extension of their student status due to health reasons according to the enrollment regulations?
6	What are the guidelines regarding the use of Generative AI (e.g., ChatGPT) in the preparation of a Master's thesis at FRI?
7	Which of the following is a criterion for a doctoral student to be awarded the 'cum laude' (with honors) distinction upon completion?
8	What is the purpose of 'Seminar 5' in the doctoral study process at FRI?
9	For first-cycle (Bachelor's) students, who must be included in the Commission for evaluation and defense?
10	In the context of language rules for a Master's thesis at FRI, what is required if a student is granted permission to write the thesis in English?
11	What are the rules regarding the citation of AI tools in academic work at the University of Ljubljana?

Appendix B: Conversational Memory Demonstration

To demonstrate the effectiveness of the query-rewriting module, the following transcript illustrates a multi-turn conversation. The user frequently omits the primary subject (e.g., 'Student Affairs Office', 'thesis') in follow-up questions, yet the system successfully maintains the conversational thread and retrieves the correct context.

User:

Kje se nahaja referat?

ReferatBot:

Študentski referat se nahaja v prostoru R1.03 na Fakulteti za računalništvo in informatiko v Ljubljani...

User:

Kdaj so odpri?

ReferatBot:

Študentski referat je odprt od ponedeljka do petka, od 9:00 do 12:00 in od 13:00 do 15:00...

User:

A jih lahko kontaktiram kadarkoli?

ReferatBot:

Ne, ne morete jih kontaktirati kadarkoli. Študentski referat je odprt od ponedeljka do petka...

User:

Koliko časa imam na voljo za izdelavo diplomskega dela?

ReferatBot:

Na podlagi navedenega konteksta imate na voljo 10 mesecev časa za izdelavo diplomskega dela od dneva prijave v študijskem informacijskem sistemu...

User:

Kaj pa za magistrsko delo?

ReferatBot:

Na podlagi navedenega konteksta imate za izdelavo magistrskega dela čas do 30. oktobra...

User:

Kaj pa če ne naredim do takrat?

ReferatBot:

Če ne oddate diplomskega dela do določenega roka, imate dve možnosti: Prošnja za podaljšanje roka... ali Ponovna prijava teme...

User:

A je isto za magisterij?

ReferatBot:

Da, pravila za magistrsko delo so podobna kot za diplomsko delo. Če ne oddate magistrskega dela do določenega roka, imate dve možnosti...